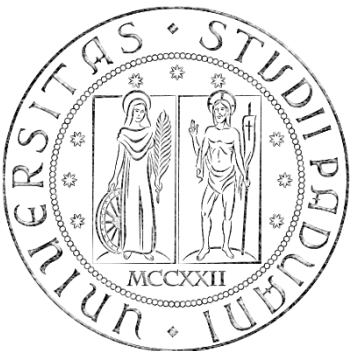


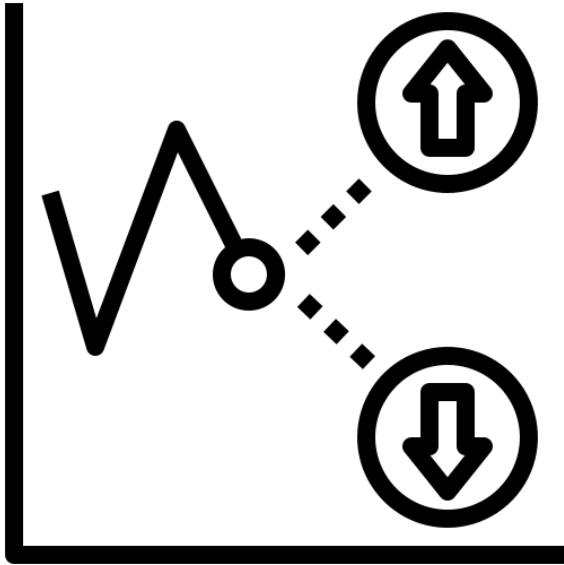
Biomedical Wearable Technologies
for Healthcare and Wellbeing

Integrating AI Models in Flutter - Part 2

A.Y. 2025-2026
Giacomo Cappon

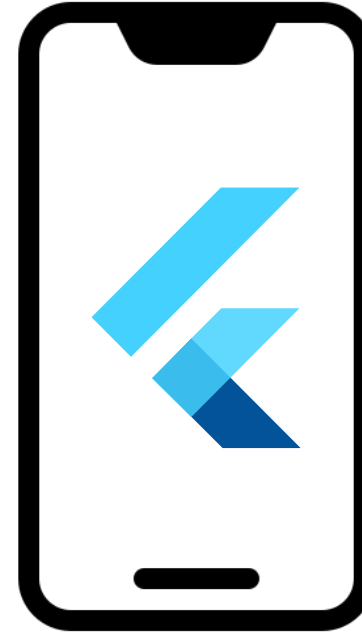


Aim



Part 1

Build a 30-minute glucose prediction model



Part 2

Integrate the model in a Flutter app

Recap: What we built in Part 1

➤ Data preparation

- ReplaceBG clinical dataset (~226 T1D patients, CGM every 5 min)
- Patient-level 80/20 train/test split to avoid data leakage
- Target: $Y[t] = \text{CGM}[t+6]$ (30 minutes ahead)

➤ Feature engineering

- Feature 1 – Current CGM value (mg/dL)
- Feature 2 – Rate of Change: $\text{ROC}[t] = (\text{CGM}[t] - \text{CGM}[t-3]) / 15$

➤ Model training & export

- sklearn pipeline: StandardScaler + LinearRegression
- Exported to `cgm_forecast.json` → ready to bundle with the Flutter app

Our model

➤ Learned prediction formula

- $Y = 58.40 * (CGM - 159.70)/64.55 + 11.84 * (ROC + 0.001)/0.952 + 159.66$

$$Y = 0.905 * CGM + 12.44 * ROC + 15.21$$

- ROC coefficient is positive: rising glucose → higher future value

Prepare the App

➤ Create the project

➤ Key source files

- `lib/main.dart` – app entry point
- `lib/models/glucose_predictor.dart` – model class
- `lib/screens/homepage.dart` – UI

➤ Bundle the model

- Place `cgm_forecast.json` in `assets/`
- Declare in `pubspec.yaml` → loaded offline at runtime

```
predict_glucose_app/  
  assets/  
    cgm_forecast.json  
  lib/  
    main.dart  
    models/  
      glucose_predictor.dart  
    screens/  
      homepage.dart  
  
# pubspec.yaml  
flutter:  
  assets:  
    - assets/cgm_forecast.json
```

Step 1: The GlucosePredictor Class

➤ Class overview

- Encapsulates model loading and inference
- Holds four parameters parsed from JSON:
- `_mean`, `_scale`, `_coef`, `_intercept`

➤ Two methods

- `load()` – async, reads & parses the JSON asset
- `predict(cgm, roc)` – runs scaled inference

➤ Design choice

- No sklearn dependency needed on the device
- Pure Dart → fast, offline, zero dependencies

```
class GlucosePredictor {  
  late List<double> _mean;  
  late List<double> _scale;  
  late List<double> _coef;  
  late double _intercept;  
  
  Future<void> load() async { ... }  
  
  double predict(double cgm,  
                 double roc) { ... }  
}
```

Step 2: Loading the Model

- Use `rootBundle` to read the bundled JSON asset
- Decode with `jsonDecode` → standard `dart:convert`
- Store lists as `List<double>` for type safety
- Called once before first prediction

```
Future<void> load() async {  
  final String raw = await rootBundle.loadString('assets/cgm_forecast.json');  
  final Map<String, dynamic> json = jsonDecode(raw);  
  
  _mean      = List<double>.from(json['mean']);  
  _scale     = List<double>.from(json['scale']);  
  _coef      = List<double>.from(json['coef']);  
  _intercept = (json['intercept'] as num).toDouble();  
}
```

Step 3: Running Predictions

- Mirrors the sklearn pipeline exactly
- StandardScaler: $x_s = (x - \text{mean}) / \text{scale}$
- Linear regression: $y = \text{coef} \cdot x_s + \text{intercept}$
- Returns predicted glucose in mg/dL (30 min ahead)

$$Y = 0.905 \cdot \text{CGM} + 12.44 \cdot \text{ROC} + 15.21$$

RMSE $\approx$ 20 mg/dL on held-out test patients

```
double predict(double cgm, double roc) {  
    final double cgmScaled = (cgm - _mean[0]) / _scale[0];  
    final double rocScaled = (roc - _mean[1]) / _scale[1];  
  
    return _coef[0] * cgmScaled + _coef[1] * rocScaled + _intercept;  
}
```

Step 4: Building the UI

- Widget: StatelessWidget
 - Simple, no local state needed
- Two TextFormField inputs
 - CGM (mg/dL) – decimal, unsigned
 - ROC (mg/dL/min) – decimal, signed
- One ElevatedButton
 - Async onPressed: load → predict → show
- Result in a Snackbar
 - “Predicted glucose in 30 min: X mg/dL”

```
onPressed: () async {  
  final double? cgm =  
    double.tryParse(_cgmController.text);  
  final double? roc =  
    double.tryParse(_rocController.text);  
  
  final predictor = GlucosePredictor();  
  await predictor.load();  
  final double pred =  
    predictor.predict(cgm!, roc!);  
  
  ScaffoldMessenger.of(context)  
    ..showSnackBar(SnackBar(  
      content:  
Text('${pred.toStringAsFixed(1)} mg/dL'  
      )))  
}
```

Run it!

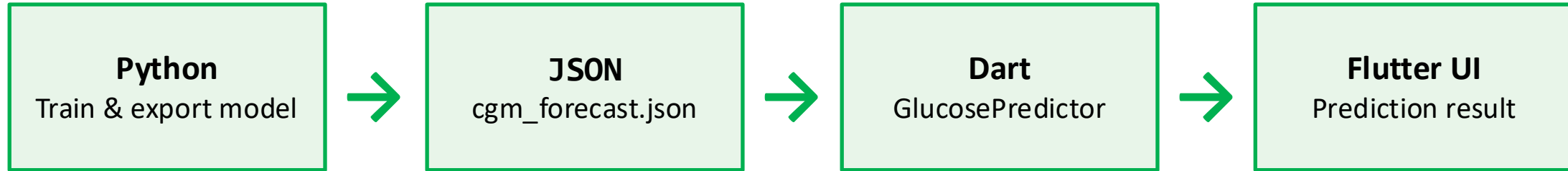
➤ End-to-end data flow

- User enters CGM and ROC values → button tap → `load()` → `predict()` → Snackbar result
- Example: CGM = 120 mg/dL, ROC = -0.5 mg/dL/min
- → Predicted glucose in 30 min: **≈ 101.5 mg/dL**

➤ Run the app

- Works fully offline – no server, no network required
- Inference is instant: pure arithmetic, no GPU needed

Putting It All Together



➤ Key takeaways

- The model was built in Python but runs entirely in Dart — no interop needed
- JSON is the portable “bridge” between the ML world and the mobile world
- This pattern scales to any model that can be expressed as a formula
- More complex models → use TFLite or ONNX runtime on the device

Open Discussion: What's Next?

- Connect to a real CGM
 - Read live BLE data from a Dexcom or FreeStyle Libre
 - Use the `flutter_blue_plus` package
- Continuous background prediction
 - Run inference every 5 minutes via a timer
 - Auto-compute ROC from the last two CGM readings
- Proactive hypoglycemia alerts
 - Push notification if predicted glucose < 70 mg/dL
 - Use `flutter_local_notifications`
- Visualize the forecast
 - Plot glucose history + predicted point on a chart
 - E.g. using the `f1_chart` package
- Upgrade to a deep learning model
 - Train an LSTM or transformer in Python
 - Export to TFLite or ONNX → run on-device with `tflite_flutter`
- Personalize the model
 - Fine-tune on individual patient data
 - On-device federated learning, privacy preserved